

Factorisation de Bunch-Parlett et Bunch-Kaufman

MTH8211 - Projet phase 1

Hortense Beaudoin et Téo Dumoutier

Description du projet

Si on tente de résoudre un système $Ax = b$ où A est dense, plusieurs solutions existent, mais une majorité sont très coûteuses. La décomposition QR et la SVD existent pour toute matrice A , y compris les matrices indéfinies (i.e il existe x t.q $x^T Ax \leq 0$ et il existe y t.q $y^T Ay \geq 0$), mais elles n'exploitent pas forcément la structure du système à résoudre.

Certaines structures symétriques sont fréquemment rencontrées lors de la résolution de problèmes. On a par exemple vu en classe qu'une autre manière d'exprimer le problème de moindre carré est de résoudre plutôt le système de point de selle

$$\begin{bmatrix} I & A \\ A^T & 0 \end{bmatrix} \begin{bmatrix} r \\ x \end{bmatrix} = \begin{bmatrix} b \\ 0 \end{bmatrix}$$

où la matrice formée de A et A^T est symétrique et toujours indéfinie. De manière intéressante, toute autre matrice B symétrique et indéfinie peut être permutée pour adopter une forme similaire, soit

$$PBP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où E est inversible. Ce type de problème est donc particulièrement fréquent et il est intéressant de s'intéresser à une méthode de résolution où la symétrie des matrices est exploitée.

La décomposition de Cholesky n'existe pas pour ce genre de système puisque la matrice considérée n'est pas définie-positive, ce qui est une exigence associée à ce type de factorisation. Une factorisation LDL^* pourrait exister, mais pas systématiquement et elle court le risque d'être numériquement instable. Higham illustre bien ce risque dans son livre avec l'exemple suivant:

$$A = \begin{bmatrix} \epsilon & 1 \\ 1 & \epsilon \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 1/\epsilon & 1 \end{bmatrix} \begin{bmatrix} \epsilon & 0 \\ 0 & \epsilon - 1/\epsilon \end{bmatrix} \begin{bmatrix} 1 & 1/\epsilon \\ 0 & 1 \end{bmatrix}$$

A est symétrique et si $\epsilon \rightarrow 0$, indéfinie. On remarque que dans le cas où $\epsilon = 0$ la factorisation n'existe tout simplement et que le reste du temps, les facteurs $1/\epsilon$ risquent de grossir de manière disproportionnée, menant à un système mal conditionné ou à des erreurs numériques importantes.

Ce projet va donc plutôt s'intéresser aux factorisations de Bunch-Parlett et Bunch-Kaufman, qui exploitent toutes deux l'idée de permutation mentionnée précédemment. La mise en place algébrique étant la même dans les deux cas, nous allons dès maintenant la présenter brièvement.

Factorisation en bloc pour matrices symétriques

Tel que mentionné ci-dessus, toute matrice $A \in \mathbb{C}^{n \times n}$ symétrique non-nulle peut être permutée de la manière suivante:

$$PAP^T = \begin{bmatrix} E & C^T \\ C & D \end{bmatrix}$$

où $E \in \mathbb{C}^{s \times s}$, $C \in \mathbb{C}^{n-s \times s}$ et $D \in \mathbb{C}^{(n-s) \times (n-s)}$ avec $s \in 1, 2$ et E inversible. Sous cette forme, il est possible de procéder à la factorisation suivante:

$$PAP^T = \begin{bmatrix} I_s & 0 \\ CE^{-1} & I_{n-s} \end{bmatrix} \begin{bmatrix} E & 0 \\ 0 & D - CE^{-1}C^T \end{bmatrix} \begin{bmatrix} I_s & E^{-1}C^T \\ 0 & I_{n-s} \end{bmatrix}$$

On reconnaît $D - CE^{-1}C^T$ comme étant le complément de Schur de la matrice PAP^T . Ce complément est aussi symétrique, par symétrie des blocs E et D ;

$$(D - CE^{-1}C^T)^T = D^T - CE^{-T}C^T = D - CE^{-1}C^T$$

On peut donc répéter le processus sur cette matrice de dimension $(n-s) \times (n-s)$, puis de nouveau et ainsi de suite jusqu'à l'obtention de la factorisation complète.

À titre d'illustration, imaginons être à l'itération k avec la forme suivante;

$$p_k A_k P_k^T = \begin{bmatrix} E_k & C_k^T \\ C_k & D_k \end{bmatrix} = \begin{bmatrix} I_{s_k} & 0 \\ C_k E_k^{-1} & I_{n-s_k} \end{bmatrix} \begin{bmatrix} E_k & 0 \\ 0 & D_k - C_k E_k^{-1} C_k^T \end{bmatrix} \begin{bmatrix} I_{s_k} & E_k^{-1} C_k^T \\ 0 & I_{n-s_k} \end{bmatrix}$$

On pose alors $A_{k+1} = D_k - C_k E_k^{-1} C_k^T$ et on procède de la même manière.

$$p_{k+1} A_{k+1} P_{k+1}^T = \begin{bmatrix} E_{k+1} & C_{k+1}^T \\ C_{k+1} & D_{k+1} \end{bmatrix} = \begin{bmatrix} I_{s_{k+1}} & 0 \\ C_{k+1} E_{k+1}^{-1} & I_{n-(s_k+s_{k+1})} \end{bmatrix} \begin{bmatrix} E_{k+1} & 0 \\ 0 & D_{k+1} - C_{k+1} E_{k+1}^{-1} C_{k+1}^T \end{bmatrix} \begin{bmatrix} I_{s_{k+1}} & E_{k+1}^{-1} C_{k+1}^T \\ 0 & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

Et ainsi de suite. Les matrices P , L et B sont construites de la façon suivante à chaque itération:

$$P_{k+1} = P_{k+1} P_k \dots P_1$$

$$L_{k+1} = \begin{bmatrix} I_{s_k} & 0 & 0 \\ (C_k E_k^{-1})_{s_{k+1}} & I_{s_{k+1}} & 0 \\ (C_k E_k^{-1})_{n-(s_k+s_{k+1})} & C_{k+1} E_{k+1}^{-1} & I_{n-(s_k+s_{k+1})} \end{bmatrix}$$

$$B_{k+1} = \begin{bmatrix} E_k & 0 & 0 \\ 0 & E_{k+1} & 0 \\ 0 & 0 & S_{k+1} \end{bmatrix}$$

La différence entre les processus de Bunch-Parlett et Bunch-Kaufman reposent dans le choix du pivot, soit de la permutation p_k . Le tout sera exploré plus en détail dans ce projet.

Objectifs

Les objectifs principaux du projet, pour débiter, sont tels que donné dans l'énoncé;

1. Comprendre et documenter les factorisations de Bunch-Parlett et Bunch-Kaufman (pivotage complet et pivotage partiel.) Fournir des exemples, des schémas explicatifs, faire des liens avec les éléments vus en classe.
 - Compréhension de base amorcée ci-dessus.
 - Le pseudo-code sera intéressant pour décrire les algorithmes à implémenter en Julia et comprendre les processus.
2. Implémenter ces deux factorisation ainsi que l'alternative du *rook pivoting*. Les comparer et identifier leurs avantages et leurs inconvénients respectifs.
 - Pour le pivotage complet, on s'attend à au plus $n^3/6$ comparaisons pour le choix de p .
 - Pour le pivotage partiel, on s'attend à $O(n^2)$ comparaisons pour le choix de p .
 - Pour le *rook pivoting*, on s'attend à entre $O(n^2)$ et $O(n^3)$ comparaisons pour le choix de p .
 - Les implémentations dans Julia nous donnerons le référent pour le nombre de flops des implémentations complètes.
3. Comparer nos implémentations à celles déjà disponibles dans Julia et tenter d'identifier les sources possibles des différences observées.
 - Julia possède une fonction de base dans LinearAlgebra appelée **BunchKaufman**. La fonction retourne trois matrices, D , U et p .
 - Nous n'avons pour l'instant pas trouvé d'implémentation de Bunch-Parlett en Julia (pas dans les fonctions de base du moins.)
4. Joindre la factorisation à des méthodes de résolution de systèmes linéaires.
5. Appliquer l'implémentation à un ensemble de problèmes modèles ainsi qu'à un problème plus spécifique au choix.
 - Nous savons que Julia possède des Packages dédiés à l'optimisation et des 'lots' de problèmes d'optimisation de base, qui peuvent être utiles pour faire du Benchmarking et comparer différentes approches. Nous verrons si ces éléments peuvent servir dans notre cas ou pas.

Plan d'action

1. Une revue de littérature sera effectuée afin de cerner l'essence de la factorisation de Bunch-Parlett et de Bunch-Kaufman et de comprendre leurs différences, leurs implications, ainsi que leurs stratégies d'implémentation. Il est attendu que le chapitre 11 du manuel *Accuracy and Stability of Numerical Algorithms* de Nicholas J. Higham soit la référence principale.
2. Une implémentation de ces deux factorisations sera mise en place. Elle sera validée à l'aide d'exemples aléatoires de divers types (nombres complexes, BigFloat, etc.) et optimisée en évaluant les performances à l'aide du module Benchmark de Julia (nombre d'allocations, temps d'exécution, etc.). Il est attendu que la documentation du module *Benchmark* de Julia se révèle utile pour savoir comment l'exploiter au maximum afin d'effectuer une analyse de performance complète.
3. La stratégie du *rook pivoting* sera implémentée comme alternative à celle du pivotage complet et partiel. Celle-ci sera validée et optimisée en utilisant les mêmes techniques que précédemment. Ses avantages et inconvénients seront documentés selon les connaissances existantes dans la littérature, ainsi que selon les observations expérimentales faites au cours de l'implémentation. Ceux-ci seront caractérisés et illustrés à l'aide d'exemples permettant de comparer ces diverses stratégies et d'identifier les différences de performance selon le cas. Encore une fois, il est attendu que le chapitre 11 du manuel *Accuracy and Stability of Numerical Algorithms* de Nicholas J. Higham soit la référence principale pour documenter les

avantages et les inconvénients de ces diverses stratégies et que la documentation du module *Benchmark* soit utile à l'analyse de performance.

4. Le tout sera formalisé afin de pouvoir être utilisé dans la résolution de systèmes linéaires en concordance avec les normes établies en Julia. Il est attendu que la documentation du module *Linear Algebra*, qui détaille ces normes, soit la référence principale afin de déterminer ce dit formalisme. On note qu'il sera nécessaire de déterminer comment utiliser les informations de cette factorisation dans le cadre de la résolution d'un système linéaire. Ceci pourrait se décliner dans les cas de résolution directe (effectuer la factorisation complète, puis s'en servir pour résoudre le système) ou itérative (effectuer progressivement la factorisation en approximant une solution de plus en plus précise au fur et à mesure). Une revue de littérature exhaustive sera faite quant à cela afin de couvrir toutes les implémentations possibles.
5. L'implémentation formalisée sera testée sur des problèmes d'optimisation quadratique divers, en évaluant les résultats et la performance. Cette évaluation sera faite par comparaison à l'implémentation de la librairie LAPACK utilisée conventionnellement en Julia. Il est attendu que la documentation de Julia et de la librairie LAPACK se révèlent utiles afin de comprendre les différences entre les implémentations et ainsi les différences entre les observations, de sorte à étoffer l'analyse et ses conclusions. On prévoit que les problèmes aux moindres carrés dans lesquels A est définie positive et de rang maximal soient un cas d'étude important, car leurs équations normales peuvent être transformées en un système de point de selle symétrique. Ces systèmes ont exactement la forme qui nous intéresse. Par ailleurs, la résolution pourra même être comparée à une résolution directe de LAPACK par factorisation QR ou encore à une résolution itérative avec LSQR, puisque ce sont des problèmes aux moindres carrés, et ce en plus de la comparaison avec une résolution LAPACK qui utilise les mêmes factorisations que celles qui seront implémentées.

Échéancier préliminaire

Sur le temps total du projet ($\sim 20h$), 15% du temps est prévu être alloué à l'étape 1, 50% à l'étape 2 et 3 qui peuvent être faites en parallèle, puisque diverses stratégies sont implémentées, 20% à l'étape 4 et 15% à l'étape 5. En effet, une portion de temps significative doit être allouée à la revue de littérature afin d'acquérir une compréhension solide sur laquelle construire le projet. De plus, il est attendu que la validation et l'optimisation de performance, qui sont des aspects importants des étapes 2 et 3, en plus de l'implémentation en tant que telle, représente un défi majeur afin d'obtenir des implémentations optimisées. Une grande portion du temps doit donc être allouée à ces étapes. Les recherches pour intégrer ces factorisations dans les différentes méthodes de résolution de système linéaire sera probablement le deuxième défi le plus important, suivi par la création de problèmes de tests robustes et adaptés. Ainsi, un peu plus du temps restant est alloué à l'étape 4 qu'à l'étape 5.

Impact attendu

Pour tout problème, il est avantageux d'exploiter la structure de ce dernier ainsi que ses symétries afin d'optimiser sa résolution. Par exemple, dans le cas d'un système linéaire $Ax = b$ dans lequel la matrice A est définie positive, la factorisation de Cholesky est utilisée à la place de l'élimination de Gauss afin d'exploiter la propriété de définie positivité pour améliorer la performance de la résolution. De la même façon, dans le cas d'un système linéaire $Ax = b$ où la matrice est hermitienne, mais pas nécessairement définie positive, ce sont les factorisations implémentées ici qui permettent d'exploiter cela afin d'améliorer la performance de la résolution. De nombreux problèmes comportent des matrices hermitiennes indéfinies, tels que des problèmes aux moindres carrés, des problèmes d'optimisation et même la résolution numérique des équations de Navier-Stokes pour fluide incompressible par exemple [1]. L'implémentation qui sera réalisée dans le cadre de ce projet est donc très utile pour tous ces types de problèmes, en notant qu'en plus leur structure possède la propriété d'améliorer la stabilité numérique des opérations de résolution [1]. Il est donc attendu que la performance (nombre d'allocations, temps d'exécution, etc.) et la stabilité numérique des problèmes à matrice hermitienne résolus grâce à ces factorisations soient bien meilleures que lorsqu'ils sont résolus avec une méthode plus générale, mais qui n'exploite pas la propriété qu'a la matrice d'être hermitienne. Par

ailleurs, les implémenter en Julia vient avec la capacité de pouvoir résoudre ces problèmes avec des types de donnée plus généraux, comme avec des *BigFloat* par exemple. En effet, autrement, Julia ne peut qu'appeler LAPACK pour utiliser ces factorisations et LAPACK ne traite pas tous les types de donnée, y compris les *BigFloat*. Ceci est donc également une contribution potentielle intéressante.

Références

[1] : Nicholas J. Higham. Accuracy and Stability of Numerical Algorithms. Society for Industrial and Applied Mathematics, second edition, 2002. doi: 10.1137/1.9780898718027. URL <https://epubs.siam.org/doi/abs/10.1137/1.9780898718027>

[Lien vers le dépôt Github du projet](#)